VIETNAM NATIONAL UNIVERSITY HO CHI MINH CITY
HO CHI MINH CITY UNIVERSITY OF TECHNOLOGY
Faculty of Electrical and Electronics Engineering

ASIC Verification

EE5213

Exercise 2 Report

FSM Reachability Analysis

Student: Pham Huy Thanh - 2570317

Instructor: TS. Tran Hoang Linh

April 1, 2026

1 Exercise 2: FSM Reachability Analysis

1.1 Objective

Implement the State Reachability Analysis algorithm for an arbitrary Finite State Machine (FSM) using C++. The program computes the set of reachable states given a set of initial states and a transition relation.

1.2 Design Specifications

- **Input:** Set of initial states (I), and a transition relation (R) mapping current states to next possible states.
- **Output:** The complete set of reachable states (ARS - All Reached States).
- **Algorithm Strategy:**
  - Track **All Reached States** (`ARS`) and **New Next States** (`NNS` or Frontier Set).
  - Iteratively compute the next states (`NS`) from the current frontier `NNS`.
  - Update the frontier as newly discovered states (`NNS` = `NS - ARS`).
  - Add the new frontier to the all reached states (`ARS` = `ARS` ∪ `NNS`).
  - Terminate when the frontier is empty.

1.3 Example FSM Analysis

To illustrate the algorithm, consider the FSM defined in the implementation with the following transition relations:

- $S_0 \to \{S_1, S_5\}$
- $S_1 \to \{S_2\}$
- $S_5 \to \{S_5, S_2\}$
- $S_2 \to \{S_3\}$
- $S_3 \to \{S_4\}$
- $S_4 \to \emptyset$

The initial state set is $I = \{S_0\}$. The state diagram is shown in Figure 1.
Step-by-Step Execution:

1. **Initialization:**
   - $ARS = \{S_0\}$
   - Frontier $NNS = \{S_0\}$
2. **Iteration 1:**
   - Compute next states from NNS: $NS = \{S_1, S_5\}$

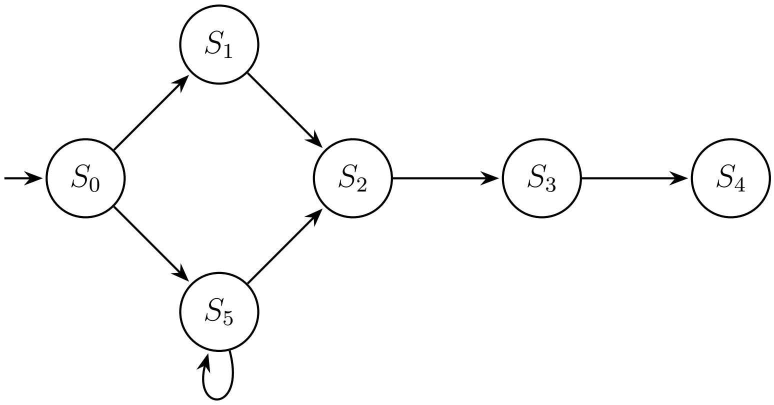

Figure 1: State Diagram of the Example FSM

- Compute new frontier: $NNS = NS \setminus ARS = \{S_1, S_5\} \setminus \{S_0\} = \{S_1, S_5\}$
- Update ARS: $ARS = ARS \cup NNS = \{S_0, S_1, S_5\}$

3. **Iteration 2:**
   - Compute next states from NNS: $NS = \{S_2, S_5\}$ (from $S_1 \to S_2$ and $S_5 \to S_2, S_5$)
   - Compute new frontier: $NNS = NS \setminus ARS = \{S_2, S_5\} \setminus \{S_0, S_1, S_5\} = \{S_2\}$
   - Update ARS: $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_5\}$

4. **Iteration 3:**
   - Compute next states from NNS: $NS = \{S_3\}$ (from $S_2 \to S_3$)
   - Compute new frontier: $NNS = NS \setminus ARS = \{S_3\} \setminus \{S_0, S_1, S_2, S_5\} = \{S_3\}$
   - Update ARS: $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_3, S_5\}$

5. **Iteration 4:**
   - Compute next states from NNS: $NS = \{S_4\}$ (from $S_3 \to S_4$)
   - Compute new frontier: $NNS = NS \setminus ARS = \{S_4\} \setminus \{S_0, S_1, S_2, S_3, S_5\} = \{S_4\}$
   - Update ARS: $ARS = ARS \cup NNS = \{S_0, S_1, S_2, S_3, S_4, S_5\}$

6. **Iteration 5:**
   - Compute next states from NNS: $NS = \emptyset$ (from $S_4 \to \emptyset$)
   - Compute new frontier: $NNS = NS \setminus ARS = \emptyset \setminus ARS = \emptyset$
   - The frontier NNS is empty, so the algorithm terminates.

Final Result: $ARS = \{S_0, S_1, S_2, S_3, S_4, S_5\}$

1.4 Implementation (fsm_reachability.cpp)

```
#include <iostream>
#include <vector>
#include <set>
#include <map>
#include <algorithm>

typedef int State;

void print_set(const std::string& name, const std::set<State>& s) {
    std::cout << name << " = { ";
    for (State val : s) std::cout << "S" << val << " ";
    std::cout << "}\n";
}

std::set<State> Reachability(
    const std::map<State, std::set<State>>& R,
    const std::set<State>& I
) {
    std::set<State> ARS = I;  // All Reached States
    std::set<State> NNS = I;  // New Next States (Frontier Set)
    int iteration = 1;

    std::cout << "--- Initialization ---\n";
    print_set("ARS", ARS);
    print_set("NNS", NNS);

    while (true) {
        std::cout << "\n--- Iteration " << iteration++ << " ---\n";
        std::set<State> NS;
        for (State s : NNS) {
            if (R.find(s) != R.end()) {
                for (State next_s : R.at(s)) {
                    NS.insert(next_s);
                }
            }
        }
        print_set("NS ", NS);

        std::set<State> new_NNS;
        std::set_difference(
            NS.begin(), NS.end(),
            ARS.begin(), ARS.end(),
            std::inserter(new_NNS, new_NNS.begin())
        );
        NNS = new_NNS;
        print_set("NNS", NNS);

        if (NNS.empty()) {
            std::cout << "NNS is empty. Terminating.\n";
            break;
        }
        ARS.insert(NNS.begin(), NNS.end());
        print_set("ARS", ARS);
    }
    return ARS;
}
```

```cpp

int main() {
    std::map<State, std::set<State>> transition_relation = {
        {0, {1, 5}}, {1, {2}}, {5, {5, 2}},
        {2, {3}}, {3, {4}}, {4, {}}
    };
    std::set<State> initial_states = {0};

    std::cout << "Starting Reachability Analysis...\n\n";
    std::set<State> reachable_states = Reachability(transition_relation, initial_states);

    std::cout << "\nFinal Reachable States (ARS): { ";
    for (State s : reachable_states) std::cout << "S" << s << " ";
    std::cout << "}\n";
    return 0;
}
```

1.5 Execution Results

```
Starting Reachability Analysis...

--- Initialization ---
ARS = { S0 }
NNS = { S0 }

--- Iteration 1 ---
NS  = { S1 S5 }
NNS = { S1 S5 }
ARS = { S0 S1 S5 }

--- Iteration 2 ---
NS  = { S2 S5 }
NNS = { S2 }
ARS = { S0 S1 S2 S5 }

--- Iteration 3 ---
NS  = { S3 }
NNS = { S3 }
ARS = { S0 S1 S2 S3 S5 }

--- Iteration 4 ---
NS  = { S4 }
NNS = { S4 }
ARS = { S0 S1 S2 S3 S4 S5 }

--- Iteration 5 ---
NS  = { }
NNS = { }
NNS is empty. Terminating.

Final Reachable States (ARS): { S0 S1 S2 S3 S4 S5 }
```